

Modelamiento de carga a la base de datos

Según el perfil de un usuario común (rol Agente)

Proyecto Ticketera Nestjs

22 de septiembre de 2026

Versión 1.0

Índice

1. Objetivo y alcance	3
2. Perfil de usuario modelado	3
3. Supuestos del modelo	3
4. De acción de usuario a consultas SQL	4
5. Carga agregada y análisis de concurrencia	5
5.1. Carga promedio del equipo	5
5.2. El problema real: ráfagas, no promedio	5
5.3. Confrontación con los límites de infraestructura	5
6. Otros perfiles, para contraste	6
7. Conclusiones y recomendaciones	6

1 Objetivo y alcance

Este documento estima la carga que un usuario típico del sistema genera sobre PostgreSQL, y la contrasta contra los límites de infraestructura ya fijados en el proyecto (pool de conexiones, *timeouts*, tamaño de lote del *scheduler* de SLA). El propósito es identificar si la configuración actual —pensada y medida, no adivinada, según `CLAUDE.md`— soporta el volumen de uso esperado, y en qué punto deja de hacerlo.

No es una medición contra tráfico real: el proyecto no tiene aún usuarios de producción, solo el *seed* de prueba (5 usuarios, ver §3). Es un modelo estimativo, con supuestos explícitos, pensado para poder recalcularse cuando existan métricas reales.

2 Perfil de usuario modelado

Se eligió el rol **AGENTE** como "usuario común" del sistema, por sobre REQUESTER y SUPERVISOR/ADMIN, por tres razones:

- Es quien más interactúa con la base durante una jornada completa: un REQUESTER típico solo abre el sistema para crear un ticket o revisar su estado —sesiones cortas y esporádicas—, mientras que el agente trabaja contra el sistema toda su jornada laboral.
- Es, numéricamente, el rol más repetido entre los usuarios activos de un sistema de mesa de ayuda: hay muchos más agentes atendiendo que supervisores o administradores.
- Es el rol que dispara la mayor variedad de operaciones sobre **tickets**: listar, ver detalle, comentar, cambiar estado — el conjunto completo de escrituras del dominio, salvo la creación (típicamente del REQUESTER o de la ingesta de correo) y la administración de usuarios (ADMIN).

Al final del documento (§6) se compara brevemente contra REQUESTER, para dimensionar cuánto pesa cada perfil en el total.

3 Supuestos del modelo

Ningún supuesto de volumen está documentado en el proyecto (el *seed* solo trae 2 agentes de prueba), así que se fijan explícitamente aquí, con un criterio conservador de institución mediana:

- Jornada de **8 horas** (28 800 segundos) por agente.
- **20 agentes** concurrentes en horario de mayor actividad — cifra asumida, ajustable; todos los cálculos de esta sección escalan linealmente con este número.
- El agente revisa su panel (*dashboard*) al iniciar turno y luego **cada 45 minutos** aproximadamente (refresco manual o vuelta a la pestaña): 10 cargas de *dashboard* por turno.
- Además del *dashboard*, hace **15 búsquedas/listados** de tickets sueltas por turno.
- Abre el **detalle de 20 tickets** por turno; en un 30 % de esos casos puede reasignar (dispara una consulta adicional a `users-service`).
- Escribe **12 comentarios** y realiza **8 cambios de estado** por turno.
- No crea tickets ni administra usuarios (eso corresponde a REQUESTER y ADMIN respectivamente).

4 De acción de usuario a consultas SQL

Cada acción del agente se traduce en una o más llamadas RPC (BFF o gateway → microservicio), y cada llamada en una o más consultas a PostgreSQL. La tabla siguiente mapea eso, a partir del código real: los *composers* del BFF (`apps/bff-web/src/composers/`) y los casos de uso de `tickets-service`.

Acción	Qué dispara	Queries/ evento	Frec./ turno	Total
Inicio de sesión	<code>users-service: SELECT</code> por email + <code>UPDATE</code> de último acceso	2	1	2
Carga de <i>dashboard</i>	<code>AgentDashboardComposer:</code> 6 llamadas RPC en paralelo (4× <code>tickets.search</code> , <code>tickets.statsByAgent</code> , <code>notifications.findForUser</code>); cada <code>search</code> pagina (consulta + <code>COUNT</code>)	10	10	100
Listado/búsqueda suelta	<code>TicketListComposer:</code> <code>tickets.search</code> + <code>COUNT</code>	2	15	30
Detalle de ticket	<code>TicketDetailComposer:</code> <code>tickets.findById</code> (con <i>joins</i>) + <code>users.find-many</code> si puede reasignar (30 % de los casos)	1,3	20	26
Comentario	Lectura de autorización + <code>INSERT ticket_comments</code> en <code>tickets-service</code> ; evento <code>ticket.commented</code> dispara en <code>notifications-service</code> resolución de destinatario + <code>INSERT notifications</code>	4	12	48
Cambio de esta- do	<code>SELECT ... FOR UPDATE</code> (lock pesimista) + <code>UPDATE</code> <code>ticket</code> + <code>INSERT</code> <code>ticket_status_history</code> ; evento <code>ticket.status-changed</code> dispara 2 queries más en <code>notifications-service</code>	5	8	40
Total por agente / turno				246

Con 246 consultas en 28 800 segundos, el promedio por agente es de **~0,0085 consultas/segundo** — una cifra trivial en sí misma. El riesgo no está en el promedio: está en la concentración de esas consultas en ráfagas cortas, que se analiza a continuación.

5 Carga agregada y análisis de concurrencia

5.1 Carga promedio del equipo

Con 20 agentes concurrentes bajo los mismos supuestos:

$$20 \times 246 = 4\,920 \text{ consultas} / \text{turno de 8h} \approx 0,17 \text{ consultas/segundo en promedio}$$

Muy por debajo de cualquier límite de `DB_POOL_MAX=10`: en régimen estacionario, el pool de 10 conexiones sobra con holgura para este volumen.

5.2 El problema real: ráfagas, no promedio

Dos eventos del sistema concentran consultas en ventanas de segundos, y son los que efectivamente presionan el pool de 10 conexiones:

1. **Inicio de turno simultáneo.** Si los 20 agentes inician sesión y cargan su *dashboard* dentro de una ventana de 2 minutos (escenario típico: turno que arranca a una hora fija), se generan $20 \times (2 + 10) = 240$ consultas en 120 segundos, con picos instantáneos de hasta $20 \times 6 = 120$ llamadas RPC casi simultáneas — cada una necesita una conexión del pool mientras dura su consulta. Con `statement_timeout=5\,000ms`, una consulta lenta retiene su conexión hasta 5 segundos; si más de 10 agentes caen en el mismo instante, las siguientes esperan en cola por una conexión libre.
2. **Corrida del *scheduler* de SLA.** Corre cada `EVERY_5_MINUTES` y procesa hasta `BATCH_SIZE=200` tickets por barrido. En un escenario realista (no los 200 completos, sino 15 tickets que efectivamente incumplen SLA en una corrida), esto dispara 1 `SELECT` de barrido + 15 `UPDATE` + 15 `INSERT` de historial + eventos a `notifications-service` (30 consultas más) — unas **46 consultas concentradas en menos de 1 segundo**, cada 5 minutos, **independientemente** de cuántos agentes estén conectados. Si esta corrida coincide con el pico de inicio de turno, ambas ráfagas compiten por el mismo pool de 10 conexiones.

Este es el mismo tipo de riesgo que `CLAUDE.md` ya documenta para `AssignTicketUseCase`: con `DB_POOL_MAX=10`, diez operaciones concurrentes que retengan una conexión agotan el pool contra sí mismas. El modelo confirma que el punto de fricción no es el volumen diario de un agente —trivial—, sino la coincidencia de ráfagas de corta duración.

5.3 Confrontación con los límites de infraestructura

Límite configurado	Valor	Lectura frente al modelo
<code>DB_POOL_MAX</code>	10 conexiones	Suficiente en régimen estacionario (0,17 consultas/seg); insuficiente si >10 operaciones de ráfaga caen en el mismo instante.
<code>statement_timeout</code>	5\,000 ms	Acota cuánto puede retener una conexión una consulta lenta durante una ráfaga; no evita la cola, solo el bloqueo indefinido.

Límite configurado	Valor	Lectura frente al modelo
lock_timeout	3 000 ms	Relevante en cambios de estado concurrentes sobre el mismo ticket (FOR UPDATE); no aplica a la carga de lectura del <i>dashboard</i> .
THROTTLE_LIMIT	100 req. / 60 000 ms (por instancia de gateway)	A nivel HTTP, 20 agentes generando ráfagas de <i>dashboard</i> (6 llamadas c/u) están muy por debajo del límite; el <i>throttling</i> no es el cuello de botella para este perfil.
BATCH_SIZE SLA	200 tickets / corrida, cada 5 min	Fuente de carga <i>independiente</i> del número de agentes conectados; debe sumarse a cualquier pico de uso humano al dimensionar el pool.

6 Otros perfiles, para contraste

- **REQUESTER**: sesión típica de 2-3 acciones (crear ticket, revisar estado, comentar) en vez de una jornada completa. Carga estimada por sesión: 1 **INSERT** (crear ticket, con su clasificación de urgencia y cálculo de SLA en la misma transacción) + 1-2 lecturas de detalle $\approx$ 4-6 consultas por sesión, muy por debajo de las 246 del agente. Aun con un volumen de REQUESTERS varias veces mayor al de agentes, su carga total es comparable o menor, porque sus sesiones son mucho más cortas y espaciadas.
- **SUPERVISOR/ADMIN**: carga por sesión más alta que la del agente cuando genera reportes o pantallas de **privacy-dossier** (portabilidad ARCO) — esta última puede disparar más de 90 consultas en una sola solicitud (hasta 50 páginas de 1 000 tickets por lote, más comentarios y notificaciones), pero es una acción infrecuente (no diaria), por lo que no domina el modelo agregado.

7 Conclusiones y recomendaciones

- El volumen *promedio* de consultas por agente ($\sim 0,0085$ consultas/seg) es irrelevante frente a la capacidad del pool actual; **no justifica, por sí solo, subir DB_POOL_MAX**.
- El riesgo real está en la **coincidencia de ráfagas**: inicio de turno simultáneo de varios agentes y la corrida del *scheduler* de SLA. Si el equipo crece más allá de los ~ 20 agentes asumidos, o si se agregan más corridas periódicas (*schedulers* adicionales), conviene volver a correr este modelo con el número real de agentes concurrentes.
- Medir esto en producción es más confiable que seguir extrapolando el supuesto de 20 agentes: instrumentar el número de conexiones activas del pool durante el inicio de turno real daría la cifra que hoy es estimada.
- Si la medición real muestra agotamiento del pool durante el pico de inicio de turno, la primera palanca es **escalonar el login** (no técnica, sino operativa: no todos los agentes

entran a la vez) antes que subir `DB_POOL_MAX`, porque este último ya tiene un costo documentado: valores altos multiplicados por réplicas de servicio agotan conexiones de PostgreSQL más rápido de lo que se gana en concurrencia (ver `CLAUDE.md`, sección de base de datos).

- Vale la pena revisar si el *scheduler* de SLA (`EVERY_5_MINUTES`) puede desplazarse fuera de los minutos típicos de inicio de turno (ej. `:02`, `:07...`) para no competir con el pico humano por el mismo pool.